

Self-Learning Report: Dockerization of the WanderLust Application

Student Name: [Your Name]

Project Name: WanderLust

Task: Dockerize the WanderLust Web Application

Date: [Submission Date]

Organization/Institute: [Organization or Institute Name]

1. Introduction

The purpose of this task was to Dockerize the WanderLust web application so that it could run in a consistent, isolated, and reproducible environment.

WanderLust is a full-stack web application developed using Node.js, Express.js, EJS, MongoDB, Passport.js, Cloudinary, and Mapbox. Before Dockerization, the application depended on locally installed software, environment variables, and a manually configured MongoDB database.

The main objective was to package the application and its dependencies into Docker containers and run the complete system through Docker Compose.

2. Project Overview

The WanderLust application allows users to:

- Register and log in.
- Create, view, edit, and delete property listings.
- Upload images for listings.
- Add and delete reviews.
- Display listing locations using maps.
- Store user sessions in MongoDB.
- Manage authentication using Passport.js.

The application uses the following technologies:

Component	Technology
Backend	Node.js and Express.js
Template engine	EJS and EJS Mate
Database	MongoDB
Object modelling	Mongoose
Authentication	Passport.js
Session storage	Connect Mongo

Component	Technology
Image storage	Cloudinary
Maps and geocoding	Mapbox
Containerization	Docker
Multi-container management	Docker Compose

3. Learning Objectives

The main learning objectives of this task were:

1. Understand the existing application architecture.
2. Identify application dependencies and external services.
3. Learn how to create a Dockerfile for a Node.js application.
4. Learn how to create and manage multiple containers using Docker Compose.
5. Connect an application container to a MongoDB container.
6. Configure environment variables securely.
7. Add container health checks.
8. Create persistent storage for MongoDB.
9. Debug Docker networking, YAML, database, and health-check issues.
10. Verify that the complete application works inside containers.

4. Initial Project Analysis

Before creating the Docker configuration, I inspected the project structure and identified the following important files and components:

- `app.js` as the main application entry point.
- `package.json` and `package-lock.json` for Node.js dependencies.
- `models/` for Mongoose schemas.
- `routes/` for Express routes.
- `controllers/` for application logic.
- `views/` for EJS templates.
- `public/` for static assets.
- `init/` for database seeding.
- `cloudConfig.js` for Cloudinary configuration.

I identified that the application required several environment variables:

```
ATLASDB_URL
SECRET
MAP_TOKEN
CLOUD_NAME
CLOUD_API_KEY
CLOUD_API_SECRET
```

For local Docker development, MongoDB credentials were also required:

```
MONGO_ROOT_USERNAME
MONGO_ROOT_PASSWORD
```

5. Problems Identified Before Dockerization

Several issues had to be addressed before the application could run reliably in containers.

5.1 Missing Start Script

The original `package.json` did not contain a proper production start command.

A start script was added:

```
"scripts": {
  "start": "node app.js",
  "seed": "node init/index.js"
}
```

This allowed Docker to start the application using:

```
npm start
```

5.2 Hard-Coded Application Port

The application was originally configured to listen on port `8080` directly.

It was updated to use environment variables:

```
const port = Number(process.env.PORT) || 8080;
const host = process.env.HOST || "0.0.0.0";
```

The server was then started using:

```
app.listen(port, host);
```

Using `0.0.0.0` was important because a server bound only to `localhost` inside a container would not be accessible from the host machine.

5.3 Database Connection Timing

The application initially started the Express server without waiting for MongoDB to connect successfully.

The startup process was updated so that the application first connects to MongoDB and then starts the server:

```

async function start() {
  await mongoose.connect(process.env.ATLASDB_URL);

  app.listen(port, host, () => {
    console.log(`Server is running on port ${port}`);
  });
}

```

This prevents the application from accepting requests before the database is available.

5.4 Incorrect Use of Localhost

The database initialization script originally used:

```

mongodb://127.0.0.1:27017/wanderlust

```

Inside Docker, `127.0.0.1` refers to the current container. It does not refer to the MongoDB container.

The correct Docker hostname is the MongoDB service name defined in Docker Compose:

```

mongo

```

The final connection string was structured as:

```

mongodb://username:password@mongo:27017/wanderlust?authSource=admin

```

This taught me that Docker Compose provides an internal DNS system where services communicate using service names.

6. Dockerfile Implementation

A multi-stage Dockerfile was created for the Node.js application.

```

# syntax=docker/dockerfile:1

FROM node:24.5.0-alpine AS dependencies

WORKDIR /usr/src/app

COPY package.json package-lock.json .npmrc ./

RUN npm ci --omit=dev && npm cache clean --force

FROM node:24.5.0-alpine AS runtime

```

```
WORKDIR /usr/src/app

ENV NODE_ENV=production
ENV PORT=8080
ENV HOST=0.0.0.0

COPY --from=dependencies /usr/src/app/node_modules ./node_modules
COPY --chown=node:node . .

USER node

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3
  CMD node -e "fetch('http://127.0.0.1:' + (process.env.PORT || 8080) + '/health').then((response) => { if (!response.ok) process.exit(1); }).catch(() => process.exit(1));"

CMD ["npm", "start"]
```

6.1 Important Dockerfile Concepts Learned

Base image

The project uses:

```
FROM node:24.5.0-alpine
```

The Alpine image is smaller than a standard Node.js image, which helps reduce the final Docker image size.

Working directory

```
WORKDIR /usr/src/app
```

This sets the default working directory inside the container.

Dependency installation

```
RUN npm ci --omit=dev
```

`npm ci` installs dependencies exactly as defined in `package-lock.json`, which provides reproducible builds.

The `--omit=dev` option excludes development dependencies from the production image.

Docker layer caching

The dependency files were copied before the complete source code:

```
COPY package.json package-lock.json .npmrc ./
RUN npm ci --omit=dev
```

This allows Docker to reuse the dependency layer when source files change but dependencies remain the same.

Non-root user

```
USER node
```

The application runs using the built-in Node.js user rather than the root user. This improves container security.

Exposed port

```
EXPOSE 8080
```

This documents the port used by the application inside the container.

7. Docker Compose Implementation

Docker Compose was used to manage the application and MongoDB containers.

The Compose configuration included two services:

1. app
2. mongo

```
services:
  mongo:
    image: mongo:8.0
    restart: unless-stopped

    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME:-wanderlust}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:-
wanderlust_dev_password}

    volumes:
      - mongo_data:/data/db

    healthcheck:
      test:
```

```

- CMD-SHELL
- >
  mongosh --quiet
  --username "${MONGO_INITDB_ROOT_USERNAME}"
  --password "${MONGO_INITDB_ROOT_PASSWORD}"
  --authenticationDatabase admin
  --eval "quit(db.adminCommand({ ping: 1 }).ok ? 0 : 2)"
interval: 10s
timeout: 5s
retries: 10
start_period: 20s

app:
  build:
    context: .
    dockerfile: Dockerfile

  restart: unless-stopped

  ports:
    - "${APP_PORT:-8080}:8080"

  env_file:
    - .env

  environment:
    NODE_ENV: production
    PORT: 8080
    HOST: 0.0.0.0
    ATLASDB_URL: mongodb://${MONGO_ROOT_USERNAME:-wanderlust}:${MONGO_ROOT_PASSWORD:-wanderlust_dev_password}@mongo:27017/wanderlust?authSource=admin

  depends_on:
    mongo:
      condition: service_healthy

  init: true

volumes:
  mongo_data:

```

8. MongoDB Container Configuration

The MongoDB service uses the official MongoDB image:

```
image: mongo:8.0
```

The database credentials are provided through environment variables:

```
environment:
  MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USERNAME}
  MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD}
```

The credentials were stored in the `.env` file:

```
MONGO_ROOT_USERNAME=wanderlust
MONGO_ROOT_PASSWORD=WanderlustLocal_2026
```

I learned that MongoDB creates the root user only during the first initialization of the database volume.

If the username or password is changed after MongoDB has already initialized, the old credentials remain in the volume. In that case, the database must be recreated using:

```
docker compose down -v
docker compose up --build -d
```

The `-v` option removes the MongoDB volume and all locally stored database data.

9. Database Persistence

A named Docker volume was used:

```
volumes:
  mongo_data:
```

The MongoDB container mounts this volume at:

```
volumes:
  - mongo_data:/data/db
```

This means MongoDB data is not lost when the container is stopped or recreated.

For example:

```
docker compose down
docker compose up -d
```

preserves the data.

However:

```
docker compose down -v
```

deletes the MongoDB volume and its data.

This helped me understand the difference between a container lifecycle and persistent data storage.

10. Health Checks

A MongoDB health check was added so that Docker could verify that the database was accepting connections.

```
healthcheck:
  test:
    - CMD-SHELL
    - >
      mongosh --quiet
      --username "${MONGO_INITDB_ROOT_USERNAME}"
      --password "${MONGO_INITDB_ROOT_PASSWORD}"
      --authenticationDatabase admin
      --eval "quit(db.adminCommand({ ping: 1 }).ok ? 0 : 2)"
```

The application service depends on MongoDB being healthy:

```
depends_on:
  mongo:
    condition: service_healthy
```

This ensures that the application does not start before MongoDB is ready.

A health endpoint was also added to the Node.js application:

```
app.get("/health", (req, res) => {
  const connected = mongoose.connection.readyState === 1;

  res.status(connected ? 200 : 503).json({
    status: connected ? "ok" : "error",
    database: connected ? "connected" : "disconnected",
    uptimeSeconds: Math.floor(process.uptime())
  });
});
```

This endpoint can be tested using:

```
curl http://localhost:8080/health
```

11. Environment Variable Management

A `.env.example` file was created to document all required variables.

```
APP_PORT=8080

MONGO_ROOT_USERNAME=wanderlust
MONGO_ROOT_PASSWORD=change_this_password

SECRET=replace_with_a_long_random_secret

MAP_TOKEN=replace_with_mapbox_token

CLOUD_NAME=replace_with_cloudinary_cloud_name
CLOUD_API_KEY=replace_with_cloudinary_api_key
CLOUD_API_SECRET=replace_with_cloudinary_api_secret

COOKIE_SECURE=false

SEED_USERNAME=demo
SEED_EMAIL=demo@example.com
SEED_PASSWORD=ChangeMe123
```

The actual `.env` file was excluded from version control because it contains secret information.

I learned that environment variables should be used instead of hard-coding:

- Database credentials.
- Session secrets.
- API keys.
- Cloud service credentials.
- Port configurations.

12. Database Seeding

The application initially displayed no listings because the MongoDB database was empty.

The seed command was used to insert sample data:

```
docker compose run --rm app npm run seed
```

The seed script connects to the same MongoDB service used by the application.

The sample listing data contains image URLs from Unsplash. Therefore:

- Listing details are stored in MongoDB.
- Sample image URLs are stored in MongoDB.
- The actual sample images remain on Unsplash.

- Sample images are not uploaded to Cloudinary.

For listings created through the web application:

- The actual image is uploaded to Cloudinary.
- The Cloudinary URL and filename are stored in MongoDB.

13. MongoDB and Cloudinary Data Flow

The final image-storage flow is:

```
User selects an image
      |
      v
Application uploads image to Cloudinary
      |
      v
Cloudinary returns URL and public identifier
      |
      v
Application stores URL and filename in MongoDB
      |
      v
Browser reads the URL from MongoDB and loads the image
```

A MongoDB listing document may contain:

```
{
  "title": "Example Listing",
  "description": "Example description",
  "price": 1200,
  "image": {
    "url": "https://res.cloudinary.com/example/image/upload/example.jpg",
    "filename": "wanderlust_DEV/example"
  }
}
```

14. Commands Used During the Task

Build and start the project

```
docker compose up --build -d
```

Check container status

```
docker compose ps
```

View application logs

```
docker compose logs -f app
```

View MongoDB logs

```
docker compose logs -f mongo
```

Validate the Compose file

```
docker compose config
```

Stop the containers

```
docker compose down
```

Stop containers and remove volumes

```
docker compose down -v
```

Restart the application

```
docker compose restart app
```

Seed the database

```
docker compose run --rm app npm run seed
```

Access the application container

```
docker compose exec app sh
```

Access MongoDB

```
docker compose exec mongo mongosh  
--username wanderlust  
--password WanderlustLocal_2026  
--authenticationDatabase admin
```

15. Challenges Faced and Solutions

Challenge 1: MongoDB container reported as unhealthy

The MongoDB container started, but Docker reported:

```
dependency failed to start: container mongo is unhealthy
```

The cause was an incorrectly written health-check command and environment variable quoting.

The health check was corrected to use container environment variables properly.

Challenge 2: YAML configuration error

Docker Compose reported:

```
services.start_period must be a mapping
```

The cause was incorrect indentation in `compose.yaml`.

The `start_period` property had to be placed under the `healthcheck` section:

```
healthcheck:
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 20s
```

This taught me that YAML is indentation-sensitive and spaces must be used consistently.

Challenge 3: Changed MongoDB password did not work

After changing the MongoDB password, authentication still failed.

The reason was that MongoDB credentials are initialized only when the volume is first created.

The issue was solved using:

```
docker compose down -v
docker compose up --build -d
```

Challenge 4: Application displayed no listings

The application started successfully, but no listings appeared.

The reason was that the new MongoDB database was empty.

The database was populated using:

```
docker compose run --rm app npm run seed
```

Challenge 5: Images were not displayed

I learned that the application uses two different image sources:

- Seed listings use external Unsplash image URLs.
- User-uploaded images use Cloudinary.

Therefore, images may not appear when:

- The database has no listings.
- A listing has no valid image URL.
- Unsplash is blocked by the network.
- Cloudinary credentials are missing or incorrect.
- The browser cannot access the external image URL.

Challenge 6: Localhost did not work between containers

The application originally expected MongoDB to be available at `127.0.0.1`.

In a containerized environment, each container has its own network namespace.

The solution was to use the Docker Compose service name:

```
mongo
```

instead of:

```
127.0.0.1
```

16. Testing and Verification

The following checks were performed after Dockerization:

1. Docker image built successfully.
2. MongoDB container started and became healthy.
3. Application container started successfully.
4. Application connected to MongoDB.
5. The health endpoint returned a successful response.
6. The application was accessible through the browser.
7. MongoDB data remained available after container restart.
8. Sample data could be inserted using the seed command.

9. Listings were displayed from MongoDB.
10. Environment variables were available inside the application container.
11. Application and MongoDB logs could be inspected.
12. Uploaded image metadata could be stored in MongoDB.
13. Cloudinary could be used for user-uploaded images.

17. Key Concepts Learned

Through this task, I learned the following important concepts.

Docker images and containers

A Docker image is a packaged template containing an application and its dependencies.

A container is a running instance of that image.

Container networking

Containers communicate using Docker networks. In Docker Compose, each service can be reached using its service name.

Persistent volumes

Containers are temporary, but named volumes can preserve database data even after containers are recreated.

Environment-based configuration

Applications should receive credentials and configuration through environment variables rather than hard-coded values.

Health checks

A running process is not always a healthy service. Health checks verify that the service is actually ready to handle requests.

Multi-container applications

Docker Compose provides a convenient way to define, start, stop, and manage multiple related containers.

Container security

Running an application as a non-root user reduces security risks.

Reproducible builds

Using a lock file and `npm ci` ensures that the same dependency versions are installed during every build.

Separation of responsibilities

The application container runs the Node.js application, while the database container runs MongoDB. External services such as Cloudinary and Mapbox remain outside the Docker environment.

18. Benefits Achieved Through Dockerization

Dockerizing the WanderLust project provided the following benefits:

- Consistent development environment.
- Reduced dependency conflicts.
- Easier project setup.
- Reproducible installation.
- Isolated application and database services.
- Simplified MongoDB setup.
- Persistent database storage.
- Easier debugging through container logs.
- Improved portability.
- Better preparation for deployment and CI/CD.
- Clear management of environment variables.
- Reliable service startup using health checks.

19. Future Improvements

The project can be improved further by implementing:

1. A development Compose configuration with source-code bind mounts.
2. Automatic restart using Nodemon during development.
3. Automated unit and integration tests.
4. GitHub Actions for image building and testing.
5. Docker image vulnerability scanning.
6. A reverse proxy using Nginx or Traefik.
7. HTTPS configuration.
8. A managed production MongoDB service.
9. Centralized logging and monitoring.
10. Kubernetes deployment configuration.
11. Better secret management.
12. Automated database backups.
13. Image fallback handling.
14. Separate development, testing, and production environments.
15. Automated deployment to a cloud container platform.

20. Conclusion

The WanderLust application was successfully Dockerized using Docker and Docker Compose.

The final environment contains a Node.js application container and a MongoDB container. Docker Compose manages networking, startup order, health checks, environment variables, port mappings, and persistent database storage.

During this task, I gained practical knowledge of Dockerfiles, Docker Compose, MongoDB authentication, container networking, persistent volumes, environment configuration, health checks, and application debugging.

The most important learning from this task was that Dockerization is not limited to writing a Dockerfile. It also requires understanding the application architecture, database behavior, service dependencies, networking, storage, security, and runtime configuration.

By completing this task, I was able to convert the WanderLust application into a portable and reproducible containerized system that can be started using a single command:

```
docker compose up --build -d
```

This task improved my understanding of both application deployment and modern DevOps practices.